

Name _____

ERP _____

Instructions:

- Attempt all questions . Understanding questions is part of quiz.
- write the final answer in the space provided for it , only coding part can be done on extra sheet.
- For each of the following program segments, assume that the code is contained within the main function and appropriate headers are also declared

PART A : DO AS DIRECTED [1 Mark Each]

1. What will be printed?

```
int numbers[] = {2, 4, 6, 8};  
int* ptr = numbers + 1;  
ptr++;  
cout << *ptr;
```

- a) 2
- b) 4
- c) 6
- d) 8

2. What happens when this code executes?

```
int* ptr = nullptr;  
cout << *ptr;
```

- a) Prints a random value
- b) Causes a runtime crash
- c) Compilation error
- d) Prints 0

3. What is the value of y after this code executes?

```
void update(int* p) {  
    static int z = 3;  
    *p += z;  
    z++; }  
int main() {  
    int y = 5;  
    update(&y);  
    update(&y);  
    cout << y; }
```

- a) 8
- b) 11
- c) 14
- d) Undefined behavior

4. What is the value of sizeof(ptr) in the following code?

```
void process(int arr[]) {  
    int* ptr = arr;  
    cout << sizeof(ptr); }  
int main() {  
    int values[10];  
    process(values); }
```

- a) Size of array (40 bytes if int is 4 bytes)
- b) Size of pointer (4 or 8 bytes)
- c) Size of one element (sizeof(int))
- d) Compilation error

5. What happens when this function executes?

```
int* allocate() {  
    int temp = 10;  
    return &temp;  
}
```

- a) Returns a valid pointer
- b) Compilation error
- c) Returns an address that leads to undefined behavior
- d) Runtime error

6. What will be printed?

```
void display() {  
    static int val = 1;  
    cout << val++ << " ";  
}  
int main() {  
    display();  
    display();}
```

- a) 1 1
- b) 1 2
- c) 2 3
- d) Undefined behavior

7. Which statement about static variables is correct?

- a) They reset their value every time a function is called
- b) They persist throughout the program's lifetime
- c) They are local variables and do not retain values
- d) They are initialized automatically as zero

8. What is wrong with this function?

```
void processArray(int arr[]) {  
    arr[10] = 50; }
```

- a) Accessing out-of-bounds memory if arr has fewer than 11 elements
- b) Memory leak
- c) Compiles but always causes a runtime error
- d) Nothing is wrong

9. What happens when this code is executed?

```
void updatePointers(int* p, int* q) {  
    int temp = *p;  
    *p = *q;  
    *q = temp;}  
int main() {  
    int a = 5, b = 10;  
    updatePointers(&a, &b);  
    cout << a << " " << b;}
```

- a) Values are swapped
- b) Pointers are swapped but values remain unchanged
- c) Compilation error
- d) Undefined behavior

10. What is the output of this code?

```
void modify(char str[]) {  
    str[0] = 'A'; }  
int main() {  
    char text[] = "Code";  
    modify(text);  
    cout << text; }
```

- a) Aode
- b) Code
- c) Compilation error
- d) Undefined behavior

11. What is the type of ptr in the following code?

```
int matrix[3][3];  
int (*ptr)[3] = matrix;
```

- a) Pointer to a 3D array
- b) Pointer to a 2D array
- c) Pointer to an array of 3 integers
- d) Array of pointers

12. What will be the output?

```
void updateStatic(int& val) {  
    static int factor = 2;  
    val += factor++;  
}  
int main() {  
    int num = 4;  
    updateStatic(num);  
    updateStatic(num);  
    cout << num;}
```

- a) 4
- b) 8
- c) 10
- d) 12

13. What happens when you execute this code?

```
const char* text = "Hello";  
text[0] = 'h';
```

- a) Modifies the string successfully
- b) Compilation error
- c) Runtime error due to modification of read-only memory
- d) Undefined behavior

14. What is the output of this code?

```
void process(int nums[], int size) {  
    static int index = 0;  
    nums[index++ % size] *= 3;  
}  
int main() {  
    int arr[] = {1, 2, 3};  
    process(arr, 3);  
    process(arr, 3);  
    cout << arr[0] << " " << arr[1];  
}
```

- a) 1 6
- b) 3 6
- c) 3 2
- d) Undefined behavior

PART B

15. Coding Part [10 Marks Each]

- 1. Write a program that finds the sum of alternate elements in an array (elements at index 0, 2, 4, etc.) using pointer arithmetic.**

```
int* getAlternateSum(int arr[], int size) {  
    // Should return static array with:  
    // result[0] = sum of elements at even indices (0,2,4...)  
    // result[1] = sum of elements at odd indices (1,3,5...)  
    // Use pointer arithmetic instead of array indexing  
}  
  
int main() {  
    int numbers[] = {1, 2, 3, 4, 5, 6};  
    int* sums = getAlternateSum(numbers, 6);  
    //Print : Sum at even indices: 9 (1+3+5)  
    // Print: Sum at odd indices: 12 (2+4+6)  
}
```

- 2. Create a function that counts vowels and consonants in a string using character array manipulation.**

```
void countLetters(const char* str, int* vowels, int* consonants) {  
    // Count vowels and consonants in str  
    // Store results in vowels and consonants  
    // Handle both uppercase and lowercase letters }  
  
int main() {  
    char text[] = "Hello World";  
    int vowelCount = 0, consonantCount = 0;  
    countLetters(text, &vowelCount, &consonantCount);  
    // Should print number of vowels and consonants }
```

- 3. Write a function that calculates the sum of each row in a 2D array and stores them in a static array.**

```
int* getRowSums(int matrix[][3], int rows) {  
    // Calculate sum of each row  
    // Return static array containing sums  
    // Use pointer arithmetic where possible }  
  
int main() {  
    int matrix[][3] = {  
        {1, 2, 3},  
        {4, 5, 6},  
        {7, 8, 9}    };  
    int* sums = getRowSums(matrix, 3);  
    // Should print: 6, 15, 24 }
```

- 4. Create two functions that share a static variable to track number of positive and negative numbers processed.**

```
void processNumber(int num) {  
    // Use static variables to count positive and negative numbers  
    // Print current counts after processing each number  
}  
  
int* getStats() {  
    // Return static array containing:  
    // [0] = count of positive numbers  
    // [1] = count of negative numbers }  
  
int main() {  
    processNumber(5); // Positive: 1, Negative: 0  
    processNumber(-2); // Positive: 1, Negative: 1  
    processNumber(7); // Positive: 2, Negative: 1  
    processNumber(-4); // Positive: 2, Negative: 2    int* stats = getStats();    // Print final statistics }
```

PART C

16. Logical Questions [20 marks]

1. [2 marks]

```
int* processArray() {  
    int localArr[] = {1, 2, 3};  
    static int staticArr[] = {1, 2, 3};  
    return staticArr; // Line A  
    return localArr; // Line B }
```

Explain why returning localArr (Line B) would cause problems while returning staticArr (Line A) works correctly. Draw the memory layout to support your answer and explain what happens to both arrays when the function returns.

2. [6 marks]

```
int matrix[3][4];  
int (*ptr)[4] = matrix;  
ptr++;
```

1. Draw and explain How is this 2D array stored in memory?
2. What does ptr++ do? How many bytes does it move?
3. Why do we need to specify the column size [4] in the pointer declaration but not the row size [3]?

3. [8 marks]

Given the following code:

```
inline int squareInline(int x) { return x * x;}  
int squareNormal(int x) { return x * x;}  
int main() {  
    int a = 4, b = 5;  
    int result1 = squareInline(a);  
    int result2 = squareNormal(b);  
    cout << "Square of " << a << " using inline function: " << result1 << endl;  
    cout << "Square of " << b << " using normal function: " << result2 << endl;  
    return 0;}
```

- a) What will be the output of the program?
- b) What is the difference in behavior between the inline function squareInline() and the normal function squareNormal()?
- c) Explain the possible performance difference between calling squareInline() and squareNormal().

5. [3 Marks]

```
void countCalls() {  
    static int count = 0;  
    int regularCount = 0;  
    count++;  
    regularCount++;  
    cout << "Static: " << count << ", Regular: " << regularCount << endl; }
```

If this function is called three times:

1. Explain the initialization and lifetime of both variables
2. What will be printed in each call and why?
3. What would happen if static initialization was: `static int count = rand() % 10;`

5. [1 Mark]

Consider these function declarations:

```
void func1(int arr[10]);  
void func2(int* arr);  
void func3(int (&arr)[10]);  
  
int main() {  
    int numbers[10] = {1,2,3,4,5,6,7,8,9,10};  
    cout << sizeof(numbers) << endl; // Line 1  
    func1(numbers);  func2(numbers);  func3(numbers); }
```

Explain:

1. Why is `sizeof(numbers)` in main different from `sizeof(arr)` inside `func1` and `func2`?

PART D

17. Complete the following function declaration to receive a 2D array of integers with 3 columns and variable number of rows. The function should return a pointer to an array containing the sum of each row.[4]

```
_____ calculateRowSums(_____) {  
    // Function implementation not needed  
}  
  
int main() {  
    int matrix[][3] = {{1,2,3}, {4,5,6}, {7,8,9}};  
    int* sums = calculateRowSums(matrix, 3); // 3 is number of rows  
}
```

18. What will be the output of the following code segment? If there's an error, explain why.

[3 Points]

```
int arr[] = {10, 20, 30, 40, 50};  
  
int* ptr = arr + 2;  
  
ptr - = 1;  
  
cout << *ptr << " " << *(ptr + 2);
```

19. What is the output of the following program? Explain the role of the static variable[4 Points]

```
void countCalls() {  
    static int count = 0;    count++;    cout << "Function called " << count << " times\n"; }  
  
int main() {  
    for(int i = 0; i < 3; i++) {  
        countCalls();    } }  
}
```

PART E

20. For each of the following code segments, specify the output or identify any errors and propose solution: [15]

a)

```
int arr[] = {1, 2, 3, 4, 5};
```

```
int* p1 = arr;
```

```
int* p2 = &arr[3];
```

```
cout << p2 - p1;
```

b)

```
void modifyArray(int* arr, int size) {
```

```
    static int lastIndex = 0;
```

```
    arr[lastIndex++] = 100;
```

```
}
```

```
int main() {
```

```
    int numbers[] = {1, 2, 3, 4};
```

```
    modifyArray(numbers, 4);
```

```
    modifyArray(numbers, 4);
```

```
    cout << numbers[0] << " " << numbers[1];
```

```
}
```

c)

```
int* createArray() {
```

```
    int arr[] = {1, 2, 3};    return arr; }
```

```
int main() {
```

```
    int* ptr = createArray();
```

```
    cout << ptr[0];}
```

d)

```
int* doubleValues(int arr[], int size) {  
    int result[10];  
    for(int i = 0; i < size; i++) {  
        result[i] = arr[i] * 2;    }    return result; }  
  
int main() {  
    int numbers[] = {1, 2, 3, 4, 5};  
    int* doubled = doubleValues(numbers, 5);  
    cout << doubled[0] << " " << doubled[1];}
```

e)

```
int* trackMax(int arr[], int size) {  
    static int maxSeen = 0;  
    for(int i = 0; i < size; i++) {  
        if(arr[i] > maxSeen)  
            maxSeen = arr[i];    }  
    return &maxSeen; }  
  
int main() {  
    int test1[] = {1, 5, 3};  
    int test2[] = {2, 4, 7};  
    int* max1 = trackMax(test1, 3);  
    cout << *max1 << endl;  
    int* max2 = trackMax(test2, 3);  
    cout << *max1 << " " << *max2; }
```